

```
# # 🚀 VLM UNIFIED PIPELINE - FACT VERIFICATION
# ## Qwen2-VL-7B-Instruct (Optimized for 2x GPU T4)

# ### Lồng xử lý:
# 1. **VLM Analysis**: Qwen2-VL-7B-Instruct phân tích ảnh + claim (few-shot + CoT)
# 2. **Atomic Facts Extraction**: Chia claim thành các fact nhỏ
# 3. **HyDE Generation**: Tạo hypothetical documents cho RAG
# 4. **Batch Processing**: Xử lý 50 samples → CSV output
# 5. **GPU Optimizations**: Device placement + Mixed precision + Gradient checkpoint
```

```
# =====
# CELL 1: INSTALLATION & IMPORTS
# =====
print("[SETUP] Installing dependencies...")
import subprocess
import sys

packages = [
    "git+https://github.com/huggingface/transformers",
    "accelerate",
    "bitsandbytes",
    "qwen-vl-utils",
    "pillow",
    "requests",
    "pandas",
    "tqdm"
]

for package in packages:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", package])

import torch
import json
import os
import re
import pandas as pd
import gc
from pathlib import Path
from tqdm import tqdm
from transformers import Qwen2VLForConditionalGeneration, AutoProcessor
from qwen_vl_utils import process_vision_info

print(f"✅ CUDA Available: {torch.cuda.is_available()}")
print(f"✅ Device Count: {torch.cuda.device_count()}")
if torch.cuda.is_available():
    for i in range(torch.cuda.device_count()):
        print(f"    GPU {i}: {torch.cuda.get_device_name(i)}")
print(f"✅ Dependencies installed!")
```

```
[SETUP] Installing dependencies...
_____ 660.6/660.6 kB 20.0 MB/s eta 0:00:00
_____ 4.2/4.2 MB 70.1 MB/s eta 0:00:00
_____ 60.7/60.7 MB 24.0 MB/s eta 0:00:00
_____ 36.3/36.3 MB 58.8 MB/s eta 0:00:00

✅ CUDA Available: True
✅ Device Count: 2
```

```
GPU 0: Tesla T4
GPU 1: Tesla T4
✅ Dependencies installed!
```

```
# =====
# CELL 2: LOAD VLM MODEL (SINGLE MODEL - XUYẾN SUỐT)
# =====
print("\n[MODEL] Loading Qwen2-VL-7B-Instruct (Optimized for Multi-GPU)...")

model_id = "Qwen/Qwen2-VL-7B-Instruct"

# 🐡 GPU Optimization: device_map="auto" for multi-GPU distribution
model = Qwen2VLForConditionalGeneration.from_pretrained(
    model_id,
    device_map="auto",
    torch_dtype=torch.float16, # Float16 để tiết kiệm memory
    # gradient_checkpointing=True # Uncomment nếu cần memory ultrasave
)
processor = AutoProcessor.from_pretrained(model_id)

print(f"✅ Model loaded: {model_id}")
print(f"✅ Model device placement: {next(model.parameters()).device}")
print(f"✅ Model dtype: {next(model.parameters()).dtype}")
```

```
[MODEL] Loading Qwen2-VL-7B-Instruct (Optimized for Multi-GPU)...
config.json: 0.00B [00:00, ?B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
model.safetensors.index.json: 0.00B [00:00, ?B/s]
Downloading (incomplete total...): 0.00B [00:00, ?B/s]
Fetching 5 files: 0%|          | 0/5 [00:00<?, ?it/s]
Loading weights: 0%|          | 0/730 [00:00<?, ?it/s]
generation_config.json: 0%|          | 0.00/244 [00:00<?, ?B/s]
preprocessor_config.json: 0%|          | 0.00/347 [00:00<?, ?B/s]
chat_template.json: 0.00B [00:00, ?B/s]
tokenizer_config.json: 0.00B [00:00, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
✅ Model loaded: Qwen/Qwen2-VL-7B-Instruct
✅ Model device placement: cuda:0
✅ Model dtype: torch.float16
```

```
# =====
# CELL 3: STRUCTURED PROMPT TEMPLATES & MULTIMODAL PREPROCESSOR CLASS
# =====

# 📄 MARKDOWN PROMPT TEMPLATES (Reusable & Professional)

VLM_IMAGE_ANALYSIS_PROMPT = """
# ROLE
You are an objective visual observer for a fact-checking system.

# TASK
Describe EXACTLY what you see in the provided image.

# CONSTRAINTS
- Focus on: people, clothing, actions, visible text, and surroundings.
- Language: Vietnamese.
```

- Format: A simple bulleted list. NO continuous paragraphs.
- NO interpretations or assumptions.

OUTPUT

(Danh sách gạch đầu dòng các sự kiện thấy trong ảnh):

"""

LLM_FACT_EXTRACTION_PROMPT = ""

ROLE

You are a logic analyst for fact verification.

TASK

Break down the claim into verifiable atomic facts.

RULES

- Each fact must be a single, complete sentence.
- Language: Vietnamese.
- Focus on: subjects, actions, locations, and time.

EXAMPLE

****Claim****: "At 10 PM yesterday, John stole a phone at the park and ran away."

****Thinking****: The claim has time (10 PM), subject (John), action 1 (stole phone), location (park), action 2 (ran away).

****Facts****:

1. Sự việc diễn ra vào lúc 10 giờ tối ngày hôm qua.
2. Đối tượng tên là John.
3. John đã thực hiện hành vi trộm điện thoại.
4. Sự việc xảy ra tại khu vực công viên.

YOUR TASK

****Claim****: "{user_claim}"

****Thinking****:

"""

LLM_JSON_RAG_PROMPT = ""

ROLE

You are a data architect for a RAG (Retrieval-Augmented Generation) system.

INPUT DATA

- ****Claim****: "{user_claim}"
- ****Image Context****: "{image_context}"
- ****Text Facts****: "{atomic_facts}"

TASK

Create a Preprocessing JSON object based on the input.

JSON FIELD INSTRUCTIONS

1. **image_facts**: Extract bullet points from Image Context.
2. **text_facts**: Extract sentences from Text Facts.
3. **normalized_facts**: Merged array of image and text facts.
4. **alignment**: Start with ONE of [Khớp hoàn toàn], [Không khớp], [Khớp một phần], then explain in 1 sentence.
5. **rag_queries**: 3 search keywords (2-5 words each) based ONLY on the Claim.
6. **hyde_doc**: 2 hypothetical documents. Doc 1: News style. Doc 2: Report style. Based ONLY on Claim.

CONSTRAINTS

- Return ONLY a valid JSON object.
- NO explanations outside JSON.
- Language: Vietnamese.

```

# OUTPUT FORMAT
{{
  "claim": "{user_claim}",
  "image_provided": {img_provided_str},
  "image_facts": [],
  "text_facts": [],
  "normalized_facts": [],
  "alignment": "",
  "rag_queries": [],
  "hyde_doc": []
}}
"""

# =====
# MULTIMODAL PREPROCESSOR CLASS (OOP Architecture)
# =====

class MultimodalPreprocessor:
    """Unified VLM processor for claims with image + text handling."""

    def __init__(self, model, processor):
        self.model = model
        self.processor = processor

    def _run_inference(self, messages, max_tokens=2048):
        """Core inference - handles both image and text inputs."""
        try:
            text = self.processor.apply_chat_template(
                messages,
                tokenize=False,
                add_generation_prompt=True
            )
            image_inputs, video_inputs = process_vision_info(messages)

            inputs = self.processor(
                text=[text],
                images=image_inputs,
                videos=video_inputs,
                padding=True,
                return_tensors="pt",
            ).to(self.model.device)

            with torch.no_grad():
                generated_ids = self.model.generate(
                    **inputs,
                    max_new_tokens=max_tokens,
                    temperature=0.3,
                    do_sample=True,
                    repetition_penalty=1.1,
                    pad_token_id=self.processor.tokenizer.pad_token_id
                )

            generated_ids_trimmed = [
                out_ids[len(in_ids):]
                for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
            ]
            output_text = self.processor.batch_decode(
                generated_ids_trimmed,

```

```

        skip_special_tokens=True,
        clean_up_tokenization_spaces=False
    )

    del inputs, generated_ids, generated_ids_trimmed
    torch.cuda.empty_cache()

    return output_text[0]

except Exception as e:
    print(f"❌ Inference error: {e}")
    return ""

def process_claim(self, claim, image_path=None):
    """Process single claim through full pipeline."""
    has_image = False
    image_context = "No image provided."

    if image_path and os.path.exists(image_path):
        has_image = True
        msg_vlm = [{"role": "user", "content": [
            {"type": "image", "image": f"file://{image_path}"},
            {"type": "text", "text": VLM_IMAGE_ANALYSIS_PROMPT}
        ]}]
        image_context = self._run_inference(msg_vlm)

    # Fact extraction
    formatted_fact_prompt = LLM_FACT_EXTRACTION_PROMPT.format(user_claim=claim)
    msg_facts = [{"role": "user", "content": [{"type": "text", "text": formatted_fact_prompt}]]
    facts_raw = self._run_inference(msg_facts)
    atomic_facts = facts_raw.split("Facts:")[1].strip() if "Facts:" in facts_raw else facts_raw

    # JSON generation
    img_str = "true" if has_image else "false"
    formatted_json_prompt = LLM_JSON_RAG_PROMPT.format(
        user_claim=claim,
        image_context=image_context,
        atomic_facts=atomic_facts,
        img_provided_str=img_str
    )
    msg_json = [{"role": "user", "content": [{"type": "text", "text": formatted_json_prompt}]]
    json_raw = self._run_inference(msg_json)

    json_match = re.search(r'\{.*\}', json_raw, re.DOTALL)
    if json_match:
        try:
            return json.loads(json_match.group(0))
        except:
            return {"error": "JSON Decode Failed", "raw": json_raw}
    return {"error": "No JSON found", "raw": json_raw}

# Initialize preprocessor
preprocessor = MultimodalPreprocessor(model, processor)
print("✅ MultimodalPreprocessor initialized!")

# Backward compatibility wrapper
def run_qwen_inference(messages, max_tokens=2048):

```

```

"""Legacy function for backward compatibility."""
return preprocessor._run_inference(messages, max_tokens)

def process_single_claim(claim_data, data_dir="/kaggle/input"):
    """
    Process a single claim using the new MultimodalPreprocessor class.
    Returns dict with all processing results.
    """
    claim_id = claim_data.get("id", "UNKNOWN")
    claim_text = claim_data.get("claims", {}).get("SUPPORTED", [{}])[0].get("claim", "")
    image_paths = claim_data.get("media", [])

    result = {
        "id": claim_id,
        "claim": claim_text,
        "image_provided": len(image_paths) > 0,
        "image_facts": [],
        "text_facts": [],
        "normalized_facts": [],
        "alignment": "",
        "rag_queries": [],
        "hyde_doc": [],
        "error": None
    }

    if not claim_text:
        result["error"] = "No claim text provided"
        return result

    try:
        # Prepare image path
        image_path = None
        if image_paths:
            image_path = os.path.join(data_dir, image_paths[0])
            if not os.path.exists(image_path):
                image_path = None

        # Use new preprocessor class
        print(f" 📄 Processing claim ID: {claim_id}...")
        json_result = preprocessor.process_claim(claim_text, image_path)

        # Map results from preprocessor output
        if "error" not in json_result:
            result["image_provided"] = json_result.get("image_provided", result["image_provided"])
            result["image_facts"] = json_result.get("image_facts", [])
            result["text_facts"] = json_result.get("text_facts", [])
            result["normalized_facts"] = json_result.get("normalized_facts", [])
            result["alignment"] = json_result.get("alignment", "")
            result["rag_queries"] = json_result.get("rag_queries", [])
            result["hyde_doc"] = json_result.get("hyde_doc", [])
        else:
            result["error"] = json_result.get("error", "Unknown error")
            result["raw_output"] = json_result.get("raw", "")
            print(f" ⚠️ Error: {result['error']}")

    except Exception as e:
        result["error"] = f"Processing error: {str(e)}"

```

```
print(f" ⚠️ {result['error']}")
```

```
return result
```

```
print("✅ Inference functions ready!")
```

```
✅ MultimodalPreprocessor initialized!
```

```
✅ Inference functions ready!
```

```
# =====
# CELL 4: LOAD TEST DATA (CSV FORMAT + MEDIA FOLDER)
# =====

# # 🛠️ Configuration
# TEST_MODE = True # Set to False to process full dataset
# NUM_TEST_SAMPLES = 600 # Number of samples for test run
# CSV_FILE = "claims_merged.csv" # CSV input file name

# 🛠️ Configuration
TEST_MODE = True # Set to False to process full dataset
START_INDEX = 1200 # Mẫu thứ 601 (index 600)
END_INDEX = 1293 # Mẫu thứ 1200 (sẽ lấy đến index 1199)
CSV_FILE = "claims_merged.csv" # CSV input file name

# Folder structure expected:
# input_folder/
#   ├── claims_merged.csv
#   └── media/
#       ├── post_1_cmt_img_0.jpg
#       ├── post_2_img_0.jpg
#       └── ...

# Try multiple possible locations for the input folder
possible_root_paths = [
    "/kaggle/input/datasets/qadeptrai123/claimvn2026", # Primary Kaggle dataset
    "/kaggle/input",
    "/kaggle/input/datasets/lctrnhu/demodataset-nlp/demodata",
    "/kaggle/input/anhnestestthu",
    ".", # Current directory
    "./demodata",
]

csv_path = None
DATA_DIR = None

for root_path in possible_root_paths:
    # Check if CSV exists in this directory
    potential_csv = os.path.join(root_path, CSV_FILE)
    if os.path.exists(potential_csv):
        csv_path = potential_csv
        DATA_DIR = root_path # Root folder containing CSV + media/
        break

# Also check subdirectories
for subdir in os.listdir(root_path) if os.path.isdir(root_path) else []:
    subdir_path = os.path.join(root_path, subdir)
    potential_csv = os.path.join(subdir_path, CSV_FILE)
    if os.path.exists(potential_csv):
```

```

        csv_path = potential_csv
        DATA_DIR = subdir_path
        break
    if csv_path:
        break

if not csv_path:
    print(f"⚠️ Could not find {CSV_FILE} in expected locations.")
    print(f"    Possible paths tried: {possible_root_paths}")
    # Create dummy data for testing
    print("    Using dummy data for testing...")
    test_data = [
        {
            "id": 170,
            "media": [],
            "claims": {
                "SUPPORTED": [{
                    "claim": "Test claim for verification"
                }]
            }
        }
    ]
]
else:
    print(f"✅ Found CSV at: {csv_path}")
    print(f"✅ Data folder (with media/): {DATA_DIR}")

    # Check for media folder
    media_folder = os.path.join(DATA_DIR, "media")
    if os.path.isdir(media_folder):
        print(f"✅ Found media folder: {media_folder}")
        media_files = os.listdir(media_folder)
        print(f"📁 Media files: {len(media_files)} images")
    else:
        print(f"⚠️ Media folder not found at: {media_folder}")
        media_folder = DATA_DIR # Fallback to root

# Helper function to convert CSV row to claim dict format
def csv_row_to_claim_dict(row):
    """Convert pandas CSV row to claim_data dict format.

    Supports:
    - 'image' column: relative path like "media/post_1_cmt_img_0.jpg"
    - Multiple images: splits by | or stores single image
    """
    image_path = row.get("image", "")
    media_list = []

    if pd.notna(image_path) and str(image_path).strip():
        # Handle multiple images separated by |
        image_paths = str(image_path).split("|")
        for img_path in image_paths:
            img_path = img_path.strip()
            if img_path:
                media_list.append(img_path)

    return {
        "id": row["id"],
        "claims": {

```



```

        "SUPPORTED": [{
            "claim": row.get("claim", "")
        }]
    },
    "media": media_list
}

# Load CSV
df = pd.read_csv(csv_path, encoding="utf-8")
print(f" 📄 CSV loaded: {len(df)} rows")
print(f" 📄 Columns: {list(df.columns)}")

# Convert CSV rows to claim dict format
test_data = []
for idx, row in df.iterrows():
    test_data.append(csv_row_to_claim_dict(row))

# # Apply test mode limit
# if TEST_MODE:
#     test_data = test_data[:NUM_TEST_SAMPLES]
#     print(f"\n 📄 TEST MODE: Processing {len(test_data)} samples")
# else:
#     print(f"\n 📄 FULL MODE: Processing {len(test_data)} samples")

# print(f"✅ Data loaded: {len(test_data)} samples ready for processing")
# Apply test mode limit
if TEST_MODE:
    test_data = test_data[START_INDEX:END_INDEX]
    print(f"\n 📄 TEST MODE: Processing {len(test_data)} samples (từ mẫu {START_INDEX + 1} đến {END_INDEX})")
else:
    print(f"\n 📄 FULL MODE: Processing {len(test_data)} samples")

print(f"✅ Data loaded: {len(test_data)} samples ready for processing")

```

```

✅ Found CSV at: /kaggle/input/datasets/qadeptrai123/claimvn2026/claims_merged.csv
✅ Data folder (with media/): /kaggle/input/datasets/qadeptrai123/claimvn2026
✅ Found media folder: /kaggle/input/datasets/qadeptrai123/claimvn2026/media
📁 Media files: 1559 images
📄 CSV loaded: 1293 rows
📄 Columns: ['id', 'claim', 'image', 'text_evidences', 'text_evidences_url', 'image_evidences', 'image_evidence_path', 'reason', 'label']

📄 TEST MODE: Processing 93 samples (từ mẫu 1201 đến 1293)
✅ Data loaded: 93 samples ready for processing

```

```

# =====
# CELL 5: BATCH PROCESSING WITH GPU OPTIMIZATION
# =====

print("\n" + "="*70)
print(" 🚀 STARTING UNIFIED VLM PIPELINE PROCESSING")
print("="*70)

results = []
errors = []

# 🔄 Progress tracking
pbar = tqdm(test_data, desc="Processing claims", unit=" sample")

```

```

for claim_data in pbar:
    try:
        # Process claim with unified VLM pipeline
        result = process_single_claim(claim_data, data_dir=DATA_DIR)
        results.append(result)

        # Update progress bar
        status = "✅" if not result.get("error") else "⚠️"
        pbar.set_description(f"{status} Processing (ID: {result['id']})")

        if result.get("error"):
            errors.append({"id": result["id"], "error": result["error"]})

        # 🧹 Memory management every 5 samples
        if len(results) % 5 == 0:
            torch.cuda.empty_cache()
            gc.collect()

    except Exception as e:
        print(f"\n❌ Error processing claim {claim_data.get('id', 'UNKNOWN')}: {e}")
        errors.append({"id": claim_data.get("id"), "error": str(e)})
        continue

pbar.close()

print("\n" + "="*70)
print(f"✅ PROCESSING COMPLETE: {len(results)} samples processed")
print(f"⚠️ Errors: {len(errors)}")
print("="*70)

```

```

✓ Processing (ID: 1273): 78% [REDACTED] | 73/93 [1:08:12<17:26, 52.32s/ sample]
✓ Processing (ID: 1274): 80% [REDACTED] | 74/93 [1:08:58<16:00, 50.55s/ sample]
✓ Processing (ID: 1275): 81% [REDACTED] | 75/93 [1:09:49<15:12, 50.72s/ sample]
⚠ Processing (ID: 1276): 82% [REDACTED] | 76/93 [1:10:27<13:15, 46.82s/ sample]
  Processing claim ID: 1277...
✓ Processing (ID: 1277): 83% [REDACTED] | 77/93 [1:11:00<11:20, 42.52s/ sample]
✓ Processing (ID: 1278): 84% [REDACTED] | 78/93 [1:11:27<09:30, 38.04s/ sample]
✓ Processing (ID: 1279): 85% [REDACTED] | 79/93 [1:12:02<08:38, 37.07s/ sample]
✓ Processing (ID: 1280): 86% [REDACTED] | 80/93 [1:12:35<07:45, 35.80s/ sample]
✓ Processing (ID: 1281): 87% [REDACTED] | 81/93 [1:13:07<06:54, 34.58s/ sample]
✓ Processing (ID: 1282): 88% [REDACTED] | 82/93 [1:14:05<07:37, 41.63s/ sample]
✓ Processing (ID: 1283): 89% [REDACTED] | 83/93 [1:15:04<07:47, 46.78s/ sample]
✓ Processing (ID: 1284): 90% [REDACTED] | 84/93 [1:15:49<06:57, 46.35s/ sample]
✓ Processing (ID: 1285): 91% [REDACTED] | 85/93 [1:16:45<06:34, 49.37s/ sample]
✓ Processing (ID: 1286): 92% [REDACTED] | 86/93 [1:17:48<06:14, 53.44s/ sample]
✓ Processing (ID: 1287): 94% [REDACTED] | 87/93 [1:18:38<05:14, 52.36s/ sample]
✓ Processing (ID: 1288): 95% [REDACTED] | 88/93 [1:19:45<04:43, 56.69s/ sample]
✓ Processing (ID: 1289): 96% [REDACTED] | 89/93 [1:20:39<03:44, 56.06s/ sample]
✓ Processing (ID: 1290): 97% [REDACTED] | 90/93 [1:21:33<02:46, 55.37s/ sample]
✓ Processing (ID: 1291): 98% [REDACTED] | 91/93 [1:22:22<01:46, 53.39s/ sample]
✓ Processing (ID: 1292): 99% [REDACTED] | 92/93 [1:23:22<00:55, 55.38s/ sample]
✓ Processing (ID: 1293): 100% [REDACTED] | 93/93 [1:24:22<00:00, 54.44s/ sample]

=====
✓ PROCESSING COMPLETE: 93 samples processed
⚠ Errors: 1
=====

```

```

  Processing claim ID: 1274...
  Processing claim ID: 1275...
  Processing claim ID: 1276...
  Error: JSON Decode Failed
  Processing claim ID: 1278...
  Processing claim ID: 1279...
  Processing claim ID: 1280...
  Processing claim ID: 1281...
  Processing claim ID: 1282...
  Processing claim ID: 1283...
  Processing claim ID: 1284...
  Processing claim ID: 1285...
  Processing claim ID: 1286...
  Processing claim ID: 1287...
  Processing claim ID: 1288...
  Processing claim ID: 1289...
  Processing claim ID: 1290...
  Processing claim ID: 1291...
  Processing claim ID: 1292...
  Processing claim ID: 1293...

```

```

# =====
# CELL 6: EXPORT RESULTS TO CSV (FIXED)
# =====

print("\n[EXPORT] Converting results to CSV format...")

# Helper function to safely join items
def safe_join(items):
    """Safely join items, converting dicts/other types to strings"""
    if not items:
        return ""
    return "|".join(str(item) for item in items)

# Flatten nested structures for CSV
csv_data = []
for result in results:
    csv_data.append({
        "id": result["id"],
        "claim": result["claim"],
        "image_provided": result["image_provided"],
        "image_facts": safe_join(result.get("image_facts")),
        "text_facts": safe_join(result.get("text_facts")),
        "normalized_facts": safe_join(result.get("normalized_facts")),
        "alignment": result.get("alignment", ""),
        "rag_queries": safe_join(result.get("rag_queries")),
        "hyde_doc_1": result.get("hyde_doc", [{}])[0] if len(result.get("hyde_doc", [])) > 0 else "",
        "hyde_doc_2": result.get("hyde_doc", [{}], {})[1] if len(result.get("hyde_doc", [])) > 1 else "",
        "error": result.get("error", "")
    })

# Create DataFrame
df = pd.DataFrame(csv_data)

```

```

# 🗑️ Multiple output locations for Kaggle compatibility
output_paths = [
    "/kaggle/working/results.csv",
    "./results.csv",
    "results.csv"
]

# Try to save to first available path
saved_path = None
for output_path in output_paths:
    try:
        # Create directory if it doesn't exist
        os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True)

        # Save with UTF-8 encoding for Vietnamese text
        df.to_csv(output_path, index=False, encoding="utf-8")
        saved_path = output_path
        break
    except Exception as e:
        print(f" ⚠️ Could not save to {output_path}: {e}")
        continue

# 📊 Summary statistics
print("\n" + "="*70)
print("📊 PROCESSING SUMMARY")
print("="*70)
print(f"✅ Total processed: {len(results)}")
print(f"✅ Successful: {len([r for r in results if not r.get('error')])}")
print(f"⚠️ With errors: {len([r for r in results if r.get('error')])}")
print(f"\n📁 CSV Columns: {' ', ' '.join(df.columns)}")
print(f"📏 CSV Shape: {df.shape[0]} rows × {df.shape[1]} columns")

if saved_path:
    print(f"\n✅ CSV EXPORTED: {saved_path}")
    print(f"   File size: {os.path.getsize(saved_path) / 1024:.2f} KB")
    print(f"   Available for download in Kaggle 'Output' panel")
else:
    print(f"\n⚠️ Could not save CSV, but data is available in 'df' variable")

# Display sample of results
print("\n" + "="*70)
print("📄 SAMPLE RESULTS (First 3 rows)")
print("="*70)
print(df[["id", "claim", "alignment", "error"]].head(3).to_string())

# 🗑️ Memory cleanup
del test_data, csv_data
torch.cuda.empty_cache()
gc.collect()

```

[EXPORT] Converting results to CSV format...

```

=====
📊 PROCESSING SUMMARY
=====
✅ Total processed: 93
✅ Successful: 92

```

⚠ With errors: 1

📄 CSV Columns: id, claim, image_provided, image_facts, text_facts, normalized_facts, alignment, rag_queries, hyde_doc_1, hyde_doc_2, error
📊 CSV Shape: 93 rows × 11 columns

✅ CSV EXPORTED: /kaggle/working/results.csv
File size: 178.84 KB
Available for download in Kaggle 'Output' panel

📄 SAMPLE RESULTS (First 3 rows)

	id	claim	alignment	error
0	1201	Các ứng viên dự tuyển viên chức chỉ cần có quốc tịch Việt Nam mà không cần cư trú tại Việt Nam.	Không khớp	None
1	1202	Thời gian tiếp nhận hồ sơ đăng ký dự tuyển viên chức kéo dài trong 30 ngày kể từ ngày ban hành thông báo.	Không khớp	None
2	1203	Mức lương khởi điểm cho vị trí Kế toán viên (hạng III) là 7.000.000 VNĐ mỗi tháng.	Không khớp	None

3780

```
# =====
# CELL 7: ADVANCED OPTIONS & NEXT STEPS
# =====

print("\n" + "="*70)
print("🔧 ADVANCED CONFIGURATION OPTIONS")
print("="*70)

# Option 1: Switch to FULL dataset (after test validation)
print("""
🚀 TO PROCESS FULL DATASET:
1. Open Cell 4 (Data Loading)
2. Change: TEST_MODE = False
3. Change: NUM_TEST_SAMPLES = 999999 (or actual dataset size)
4. Re-run from Cell 4 onwards

Example code snippet:
```python
TEST_MODE = False # Process all data
NUM_TEST_SAMPLES = 999999 # No limit
```
""")

# Option 2: Memory optimization for larger batches
print("""
🚀 MEMORY OPTIMIZATION (if CUDA OOM errors):
1. Open Cell 2 (Model Loading)
2. Uncomment: # gradient_checkpointing=True
3. Or add after processor loading:
```python
model.gradient_checkpointing_enable() # Extra memory savings
```

3. Reduce inference max_tokens in Cell 5:
```python
In process_single_claim(), change:
run_qwen_inference(msg, max_tokens=256) # Instead of 1024
```
""")

# Option 3: Try other VLM models
print("""
```

```

print(
    """
    🚀 TO TEST OTHER VLM MODELS:
    1. Open Cell 2 (Model Loading)
    2. Change model_id to one of:

    ✅ Recommended alternatives:
    - "Qwen/Qwen2-VL-2B-Instruct" (Smaller, faster, 2B params)
    - "Qwen/Qwen2.5-VL-32B-Instruct" (Larger, better quality, 32B params)
    - "llama-vision/llama-3.2-11B-Vision-Instruct"
    - "microsoft/Phi-3.5-Vision-Instruct"

    3. Re-run Cell 2 and Cell 3 to reload
    """
)

# Option 4: Export formats
print("""
    🚀 EXPORT OPTIONS:
    ✅ Current: results.csv (UTF-8 encoded for Vietnamese)

    To add JSON export in Cell 6, add:
    ```python
 # Save as JSON
 with open("/kaggle/working/results.json", "w", encoding="utf-8") as f:
 json.dump(results, f, ensure_ascii=False, indent=2)
    ```

    To save as Parquet (compressed):
    ```python
 df.to_parquet("/kaggle/working/results.parquet")
    ```
    """)

# Option 5: GPU monitoring
print("""
    🚀 GPU MONITORING:
    Run this cell to check GPU usage during processing:
    ```python
 import subprocess
 subprocess.run(["nvidia-smi"], check=False)
    ```
    """)

print("\n" + "="*70)
print("✅ PIPELINE READY FOR PRODUCTION")
print("="*70)
print("\n📌 NEXT STEPS RECOMMENDED:")
print(" 1. ✅ Validate CSV output manually (first 50 samples)")
print(" 2. ✅ Share results.csv with team for review")
print(" 3. ✅ If OK → Set TEST_MODE=False for full dataset")
print(" 4. ✅ Try other VLM models from recommendations above")
print("="*70)

```

```

=====
🔧 ADVANCED CONFIGURATION OPTIONS
=====

🚀 TO PROCESS FULL DATASET:
1. Open Cell 4 (Data Loading)

```

2. Change: TEST_MODE = False
3. Change: NUM_TEST_SAMPLES = 999999 (or actual dataset size)
4. Re-run from Cell 4 onwards

Example code snippet:

```
```python
TEST_MODE = False # Process all data
NUM_TEST_SAMPLES = 999999 # No limit
```
```

✦ MEMORY OPTIMIZATION (if CUDA OOM errors):

1. Open Cell 2 (Model Loading)
 2. Uncomment: # gradient_checkpointing=True
 3. Or add after processor loading:
- ```
```python
# model.gradient_checkpointing_enable() # Extra memory savings
```
```

3. Reduce inference max\_tokens in Cell 5:

```
```python
# In process_single_claim(), change:
run_qwen_inference(msg, max_tokens=256) # Instead of 1024
```
```

#### ✦ TO TEST OTHER VLM MODELS:

1. Open Cell 2 (Model Loading)
2. Change model\_id to one of:

- ✓ Recommended alternatives:
- "Qwen/Qwen2-VL-2B-Instruct" (Smaller, faster, 2B params)
- "Qwen/Qwen2.5-VL-32B-Instruct" (Larger, better quality, 32B params)
- "llama-vision/llama-3.2-11B-Vision-Instruct"
- "microsoft/Phi-3.5-Vision-Instruct"

3. Re-run Cell 2 and Cell 3 to reload

#### ✦ EXPORT OPTIONS:

- ✓ Current: results.csv (UTF-8 encoded for Vietnamese)

To add JSON export in Cell 6, add:

```
```python
# Save as JSON
with open("/kaggle/working/results.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
```
```

To save as Parquet (compressed):

```
```python
```

